Pemrograman Berorientasi Object

Polimorfisme

Team Teaching PBO | Jurusan Teknologi Informasi

Outline Materi

Bagian I: Fondasi & Mekanisme

- **1. Definisi & Konsep Dasar:** Makna kata, analogi, dan peran superclass Object
- **2. Static Polymorphism:** Method Overloading, aturan signature, compile-time binding
- **3. Dynamic Polymorphism:** Method Overriding, late binding, referensi superclass
- **4. Virtual Method Invocation:** Mekanisme JVM dynamic dispatch & vtable
- **5. Polymorphic Arguments:** Parameter fleksibel bertipe parent class
- **6. Heterogeneous Collection:** Array & koleksi beragam objek turunan

Bagian II: Penerapan & Latihan

- **7. Polimorfisme Interface:** Kontrak perilaku melintasi rumpun hierarki
- **8. Operator instanceof:** Pemeriksaan tipe objek secara aman saat runtime
- **9. Object Casting:** Aturan Upcasting vs bahaya Downcasting (ClassCastException)
- **10. Modern Java:** Pattern Matching for instanceof (Java 16+)
- **11. Jebakan OOP:** Variable Hiding & batasan method non-virtual
- **12. Latihan Praktikum:** Exercise 1 (Penggajian) & Exercise 2 (Elektronik)

Definisi Polimorfisme

Konsep Fundamental

- **Etimologi:** Berasal dari bahasa Yunani kuno: **Poly** (Banyak) dan **Morphe** (Bentuk). Secara harfiah berarti *mempunyai banyak bentuk*.
- **Definisi PBO:** Polimorfisme adalah kemampuan suatu objek atau antarmuka untuk memiliki banyak bentuk implementasi perilaku yang berbeda.
- **Esensi Dasar:** Objek dari subclass dapat diperlakukan sebagai objek dari superclass-nya, namun tetap mempertahankan perilaku khas dari subclass tersebut saat dieksekusi.
- **Tujuan Utama:** Meningkatkan fleksibilitas dan ekstensibilitas kode program, mengurangi redundansi, serta memfasilitasi arsitektur perangkat lunak yang *loose coupling*.

Prinsip Kunci Polimorfisme

Satu antarmuka umum (common interface) dapat digunakan untuk mengontrol sekumpulan aksi yang berbeda secara spesifik di balik layar. Polimorfisme memungkinkan kita menulis kode yang berorientasi pada generalisasi, bukan keterikatan pada detail spesifik.

Semua Class di Java Bersifat Polimorfik

Hierarki Fundamental Java

• Di Java, **semua class tanpa terkecuali** secara otomatis merupakan turunan (subclass) dari class induk tertinggi: **`java.lang.Object`**.

• Bahkan ketika kita mendeklarasikan class baru tanpa kata kunci `extends`, Java compiler secara implisit menambahkan `extends Object`.

• **Dampaknya:** Setiap objek yang kita ciptakan di Java memiliki setidaknya **dua bentuk**: bentuk tipe kelasnya sendiri dan bentuk tipe `Object`.

• Sebagai analogi: Terdapat class `Binatang` dan class `Kucing` (dimana `Kucing extends Binatang`). Kita dapat menyatakan secara pasti bahwa: *'Semua kucing adalah binatang'*, namun *'Tidak semua binatang adalah kucing'*.

Contoh Validitas Polimorfik

```
// Semua deklarasi di bawah ini VALID di Java:
Kucing ciko = new Kucing();
Binatang b = ciko; // Valid (Kucing adalah Binatang)
Object obj = ciko; // Valid (Kucing adalah Object)

// Array Object dapat menampung tipe apa saja:
Object[] sembarang = {
    new Kucing(),
    "Polinema",
    Integer.valueOf(100),
    new Binatang()
};
```

Analogi Polimorfisme dalam Dunia Nyata

Skenario Manusia & Buah

- **Skenario:** Manusia dapat memakan berbagai macam buah (`Buah`), seperti `Apel`, `Pisang`, `Jeruk`, atau `Semangka`.
- **Karakteristik Umum:** Apel dan Pisang sama-sama tergolong Buah dan keduanya dapat dimakan.
- **Perbedaan Perilaku:** Cara memakannya berbeda secara spesifik: Apel langsung digigit/dicuci, sedangkan Pisang harus dikupas kulitnya terlebih dahulu.
- **Representasi Program:** Class `Manusia` cukup menyediakan satu method: `makanBuah(Buah b)`. Parameter `b` sanggup menerima objek turunan Buah mana pun tanpa perlu membuat method terpisah seperti `makanApel()`, `makanPisang()`, dsb.

Representasi Kode

```
public class Manusia {
    // Menerima parameter polimorfik Buah
    public void makanBuah(Buah b) {
        // Objek b diperlakukan sesuai
bentuk aslinya
        if (b instanceof Pisang) {
            System.out.println("Kupas
kulitnya dulu...");
        } else if (b instanceof Apel) {
            System.out.println("Cuci dan
langsung gigit!");
        }
    }
}
```

Jenis-Jenis Polimorfisme

Kriteria	Static Polymorphism	Dynamic Polymorphism
Istilah Lain	Compile-Time Polymorphism / Static Binding	Runtime Polymorphism / Dynamic Binding
Implementasi	Method Overloading	Method Overriding
Waktu Penentuan	Saat kompilasi source code (.java -> .class)	Saat program sedang dieksekusi (di JVM)
Kebutuhan Relasi	Dalam satu class (tidak perlu inheritance)	Wajib ada hubungan Inheritance / Interface
Penentu Eksekusi	Signature method (nama, jumlah/tipe parameter)	Tipe objek aktual di memori Heap saat runtime
Kecepatan	Sangat cepat (tanpa runtime overhead)	Sangat fleksibel (menggunakan lookup vtable)

Kapan Menggunakan Masing-Masing?

Gunakan Static Polymorphism ketika operasi yang sama memerlukan variasi tipe/jumlah input data yang berbeda dalam satu entitas. Gunakan Dynamic Polymorphism ketika entitas turunan memiliki implementasi perilaku yang berbeda dari entitas induknya.

Static Polymorphism (Method Overloading)

Karakteristik & Aturan Overloading

• Terjadi ketika terdapat **beberapa method dengan nama yang sama** di dalam satu class, namun memiliki **parameter / signature yang berbeda**.

• **Aturan Penentuan Signature Berbeda:**

• 1. Jumlah parameter berbeda (contoh: 2 parameter vs 3 parameter)

• 2. Tipe data parameter berbeda (contoh: `int` vs `double`)

• 3. Urutan tipe data parameter berbeda (contoh: `(int, double)` vs `(double, int)`)

• **Penting:** Perbedaan hanya pada *return type* (tipe nilai balik) atau *access modifier* **TIDAK** dianggap overloading dan akan menyebabkan compiler error!

• Compiler Java sudah mengetahui secara pasti method mana yang akan dipanggil dengan cara mencocokkan argumen saat kompilasi (*compile-time binding*).

Contoh Static Polymorphism: DemoOverload

Class DemoOverload.java

```java
public class DemoOverload {
    // Varian 1: dua parameter integer
    public int add(int x, int y) {
        return x + y;
    }
    // Varian 2: tiga parameter integer
    public int add(int x, int y, int z) {
        return x + y + z;
    }
    // Varian 3: double dan int
    public int add(double x, int y) {
        return (int) x + y;
    }
    // Varian 4: int dan double
    public int add(int x, double y) {
        return x + (int) y;
    }
}
```

Class TestOverload.java

```java
public class TestOverload {
    public static void main(String[] args) {
        DemoOverload demo = new DemoOverload();

        // Compiler mencocokkan signature:
        System.out.println(demo.add(2, 3));       //
Varian 1 -> 5
        System.out.println(demo.add(2, 3, 4));    //
Varian 2 -> 9
        System.out.println(demo.add(2, 3.4));     //
Varian 4 -> 5
        System.out.println(demo.add(2.5, 3));     //
Varian 3 -> 5
    }
}
```

Catatan Eksekusi Compiler

Saat baris 'demo.add(2, 3.4)' dikompilasi, compiler melihat parameter ke-1 adalah int (2) dan parameter ke-2 adalah double (3.4). Compiler langsung mengikat pemanggilan ke Varian 4 tanpa perlu menunggu program dijalankan.

Dynamic Polymorphism (Method Overriding)

Mekanisme Referensi Polimorfik

• Dynamic Polymorphism terjadi saat ada objek bertipe **parent-class** yang memegang referensi ke instance **sub-class**, dan ada overriding method yang dipanggil.

• **Sintaks Deklarasi Polimorfik:**

• `Employee emp = new Manager();`

• • `Employee` : Tipe referensi (compile-time type / deklarasi)

• • `Manager` : Tipe objek sesungguhnya di memori heap (runtime type)

• • Di mana class `Manager` adalah subclass dari `Employee`.

• Objek `emp` dideklarasikan bertipe `Employee`, namun memegang referensi instance `Manager`.

• Saat method overriding dipanggil melalui `emp`, method yang dijalankan oleh JVM adalah versi milik **`Manager`**!

Contoh Dynamic Polymorphism: Vehicle & MotorBike

Superclass & Subclass

```
public class Vehicle {
    public void move() {
        System.out.println("Vehicles can move!!");
    }
}

public class MotorBike extends Vehicle {
    @Override
    public void move() {
        System.out.println("MotorBike can move and
accelerate too!!");
    }
}
```

Class TestMotorBike.java

```
public class TestMotorBike {
    public static void main(String[] args) {
        // vh bertipe Vehicle, referensi ke MotorBike
        Vehicle vh = new MotorBike();
        vh.move(); // Menjalankan MotorBike.move()!

        // Mengembalikan referensi vh ke Vehicle
        vh = new Vehicle();
        vh.move(); // Menjalankan Vehicle.move()!
    }
}
```

Dynamic Polymorphism: Hasil & Analisis Eksekusi

Command Prompt / Terminal

```
D:\Java\Polimorfisme> javac TestMotorBike.java
D:\Java\Polimorfisme> java TestMotorBike
MotorBike can move and accelerate too!!
Vehicles can move!!
```

Penjelasan Alur Program

- Pertama, objek `vh` dibuat bertipe `Vehicle`, namun memegang referensi instance class `MotorBike`.
- Pada pemanggilan `vh.move()` pertama, JVM mengeksekusi method `move()` milik class `MotorBike`.
- Kemudian objek `vh` di-reassign referensinya ke objek murni bertipe `Vehicle` (`vh = new Vehicle()`).
- Setelah itu, pemanggilan `vh.move()` kedua mengeksekusi method `move()` milik class `Vehicle`.
- **Kesimpulan:** Perilaku method `move()` berubah secara dinamis pada saat program berjalan (*runtime*) sesuai objek yang sedang direferensikan.

Konsep Kunci: Virtual Method Invocation (VMI)

Mekanisme Internal JVM

• **Apa itu VMI?** Virtual Method Invocation terjadi ketika ada pemanggilan overriding method dari suatu objek polimorfisme.

• **Mengapa disebut 'Virtual'?** Karena terdapat perbedaan antara method yang dikenali oleh compiler saat compile-time dengan method yang benar-benar dijalankan oleh JVM saat runtime.

• **Compile-Time:** Compiler hanya memeriksa apakah method tersebut *terdaftar* pada tipe referensi (`Vehicle`). Jika ada, kompilasi berhasil.

• **Run-Time:** JVM menggunakan mekanisme *Dynamic Method Dispatch* (tabel `vtable`) untuk menemukan implementasi method pada class objek aktual di memori heap (`MotorBike`).

• **Karakteristik Java:** Di Java, seluruh non-static method secara default adalah *virtual method* (berbeda dengan C++ yang memerlukan kata kunci `virtual`).

Ilustrasi Pemanggilan VMI

```
Animal a = new Deer();

// Compile-Time:
// Compiler mencocokkan a.walk()
// dengan definisi pada class Animal.

// Run-Time:
// JVM mendeteksi objek aktual adalah Deer.
// JVM mengeksekusi Deer.walk()!
a.walk();
```

Catatan Penting

VMI hanya berlaku untuk method overriding. Atribut (field) dan method static TIDAK bersifat virtual!

Polymorphic Arguments

Konsep & Manfaat

• **Definisi:** Parameter suatu method yang dideklarasikan dengan tipe superclass (atau interface), sehingga dapat menerima nilai argumen berupa objek dari sub-class mana pun.

• **Kekuatan Polymorphic Arguments:**

• • **Prinsip Generalisasi**: Method dirancang untuk berinteraksi dengan tipe umum, bukan terikat pada subclass tertentu.

• • **Menghindari Duplikasi Kode**: Tidak perlu membuat method yang di-overload secara manual untuk setiap kemungkinan subclass baru.

• • **Kemudahan Perawatan (Maintainability)**: Ketika di kemudian hari ditambahkan subclass baru, method penerima tidak perlu diubah sama sekali (memenuhi *Open/Closed Principle*).

Pola Desain Universal

Polymorphic argument adalah salah satu pilar utama arsitektur Java modern. Contohnya pada pustaka standar Java: method 'System.out.println(Object obj)' menerima parameter bertipe Object, sehingga ia mampu mencetak objek dari tipe apapun di dunia Java!

Contoh Polymorphic Arguments: Pegawai & Manajer

Implementasi Kode

```
public class Pegawai {
    // some class members
}

public class Manajer extends Pegawai {
    // some class members
}

public class Test {
    // Method Proses memiliki parameter bertipe Pegawai
    public static void Proses(Pegawai peg) {
        // Melakukan pemrosesan umum pegawai...
    }

    public static void main(String[] args) {
        Manajer man = new Manajer();
        // Parameter bertipe Pegawai diberi objek bertipe
Manajer:
        Proses(man); // VALID & POLIMORFIK!
    }
}
```

Penjelasan Teknis

- Class `Manajer` adalah sub class dari class `Pegawai`.
- Perhatikan class `Test`: terdapat method `Proses()` yang menerima parameter bertipe `Pegawai`.
- Pada saat method `Proses(man)` dipanggil, parameternya diisi dengan objek `man` yang sesungguhnya bertipe `Manajer`.
- Hal ini sah dan valid karena relasi pewarisan menyatakan bahwa setiap `Manajer` IS-A `Pegawai`.
- Method `Proses()` juga otomatis dapat menerima objek dari subclass lain seperti `Programmer`, `Kurir`, atau `Direktur`.

Heterogeneous Collection (Koleksi Polimorfik)

Konsep Heterogeneous Collection

• Dalam Java murni, struktur data array (`[]`) bersifat *homogen* (hanya dapat menampung elemen dengan tipe data yang sama persis).

• Namun melalui **polimorfisme**, kita dapat menciptakan **Heterogeneous Collection** (koleksi yang menampung beragam objek berbeda bentuk) dengan cara mendeklarasikan tipe array menggunakan superclass atau interface.

• Array bertipe superclass dapat diisi dengan objek dari berbagai macam sub-class turunannya.

• **Manfaat Nyata:**

• • Memungkinkan pemrosesan data massal secara seragam (*batch processing*).

• • Mengurangi percabangan kode yang rumit dan panjang.

• • Menyatukan manajemen memori koleksi objek dalam satu variabel referensi.

Implementasi Heterogeneous Collection

Kode Heterogeneous Collection

```
// Membuat array polimorfik bertipe superclass Animal:
Animal[] daftarHewan = new Animal[3];

// Mengisi array dengan instance objek dari berbagai
subclass:
daftarHewan[0] = new Deer();
daftarHewan[1] = new Lion();
daftarHewan[2] = new Deer();

// Memproses seluruh elemen secara seragam dengan satu
perulangan:
for (Animal a : daftarHewan) {
    a.walk(); // Dynamic dispatch memanggil method spesifik
tiap hewan!
}
```

Command Prompt / Terminal

```
D:\Java> java TestHeteroCollection
The deer is walking around the jungle
The lion is hunting across the savanna
The deer is walking around the jungle
```

Elegansi Looping Polimorfik

Perhatikan perulangan for-each di atas: tidak ada satupun percabangan 'if (hewan == Deer)'. JVM secara mandiri memanggil perilaku yang tepat untuk setiap elemen saat runtime!

Polimorfisme Berbasis Interface

Kekuatan Polimorfisme Interface

• Polimorfisme tidak hanya terbatas pada pewarisan kelas tunggal (`extends`), tetapi sangat luas dan fleksibel saat menggunakan **Interface** (`implements`).

• **Mengapa Interface Lebih Fleksibel?**

• • Java hanya mendukung *Single Inheritance* untuk class (satu class hanya boleh memiliki satu induk).

• • Suatu class dapat mengimplementasikan **banyak interface** sekaligus (*multiple implementation*).

• • Interface bertindak sebagai **kontrak perilaku** (*behavioral contract*) yang menyatukan class-class yang sama sekali tidak memiliki hubungan darah silsilah.

• Objek yang dideklarasikan bertipe interface dapat mereferensikan instance dari class manapun yang meng-implement interface tersebut.

Studi Kasus Interface: Sistem Payable

Interface & Implementasi Class

```
public interface Payable {
    int getPaymentAmount();
}

// Pegawai tetap adalah karyawan DAN dapat dibayar
public class PermanentEmployee extends Employee implements Payable
{
    @Override
    public int getPaymentAmount() { return 5000000; }
}

// Tagihan listrik BUKAN karyawan, tapi sama-sama dapat dibayar!
public class ElectricityBill implements Payable {
    @Override
    public int getPaymentAmount() { return 750000; }
}
```

Pemrosesan Terpadu

```
// Keduanya dikelola bersama via Payable:
Payable[] kewajiban = {
    new PermanentEmployee("Budi", 5000000),
    new ElectricityBill("PLN-0123", 750000)
};

int totalPengeluaran = 0;
for (Payable p : kewajiban) {
    totalPengeluaran += p.getPaymentAmount();
}
System.out.println("Total: " + totalPengeluaran);
```

Operator instanceof

Mekanisme & Sintaks

• **Fungsi:** Operator biner yang digunakan untuk menguji apakah suatu objek referensi merupakan hasil instansiasi dari tipe class tertentu atau sub-class-nya.

• **Hasil Evaluasi:** Menghasilkan nilai boolean: **`true`** atau **`false`**.

• **Sintaks Dasar:**

• `if (objekRef instanceof NamaClass) { ... }`

• **Kapan instanceof Dibutuhkan?**

• • Saat menerima *polymorphic arguments*, kita seringkali perlu mengetahui tipe spesifik objek sebelum memanggil method khusus yang hanya ada di subclass.

• **Aturan Keamanan:** Jika `objekRef` bernilai `null`, operator `instanceof` akan selalu menghasilkan `false` (tidak menyebabkan NullPointerException).

Penerapan Operator instanceof

Pengecekan Tipe Polimorfik

```
public class Pegawai { /* member umum */ }
public class Manajer extends Pegawai { /* tugas manajer */ }
public class Kurir extends Pegawai { /* tugas kurir */ }

public class Test {
    public static void Proses(Pegawai peg) {
        if (peg instanceof Manajer) {
            // Dilakukan jika objek sesungguhnya adalah Manajer
            System.out.println("Menjalankan tugas Manajer...");
        } else if (peg instanceof Kurir) {
            // Dilakukan jika objek sesungguhnya adalah Kurir
            System.out.println("Menjalankan tugas Kurir...");
        } else {
            // Dilakukan untuk pegawai umum lainnya
            System.out.println("Menjalankan tugas umum pegawai...");
        }
    }

    public static void main(String[] args) {
        Manajer man = new Manajer();
        Kurir kur = new Kurir();
        Proses(man); // Masuk blok if Manajer
        Proses(kur); // Masuk blok else-if Kurir
    }
}
```

Alur Logika

- Parameter `peg` bertipe superclass `Pegawai`.
- Saat runtime, `peg instanceof Manajer` akan bernilai `true` jika argumen yang dilewatkan dibuat dari `new Manajer()`.
- Dengan mekanisme ini, method `Proses` dapat membedakan alur kerja spesifik berdasarkan identitas kelas asli dari objek tersebut.
- Pengecekan ini adalah langkah awal wajib sebelum melakukan konversi tipe (*Object Casting*).

Object Casting: Upcasting vs Downcasting

1. Upcasting (Penyempitan Detail)

• Mengubah referensi objek dari **subclass** menjadi **superclass**.

• **Bersifat Implisit (Otomatis)** dan **selalu aman** (*type-safe*).

• Terjadi secara otomatis saat penugasan objek subclass ke variabel superclass.

• Contoh: `Deer d = new Deer(); Animal a = d;`

2. Downcasting (Pemulihan Detail)

• Mengubah referensi objek dari **superclass** kembali menjadi **subclass** aslinya.

• **Wajib Eksplisit:** Tipe tujuan harus ditulis dalam kurung: `(Subclass) obj`.

• **Tujuan:** Agar dapat mengakses method & atribut spesifik subclass yang tidak ada di superclass.

• Contoh: `Deer d = (Deer) a;`

Sintaks Object Casting

```
// 1. Upcasting (Aman & Otomatis)
Pegawai p = new Manajer(); // referensi p hanya bisa mengakses member Pegawai

// 2. Downcasting (Eksplisit untuk mengakses member spesifik Manajer)
if (p instanceof Manajer) {
    Manajer m = (Manajer) p; // Explicit cast
    m.getTunjangan();        // Sah! Method spesifik Manajer kini dapat diakses
}
```

Bahaya Downcasting & ClassCastException

Ilustrasi Kesalahan Downcasting

```
Pegawai p = new Programmer("Budi", 600, 30);

// KESALAHAN FATAL: Memaksa casting Programmer menjadi
Manajer!
// Compiler meloloskan kode ini karena p bertipe Pegawai...
Manajer m = (Manajer) p;

// Namun saat dijalankan, JVM akan langsung CRASH:
// Exception in thread "main" java.lang.ClassCastException:
// class Programmer cannot be cast to class Manajer
```

Analisis & Pencegahan

• **Mengapa Compiler Tidak Protes?**

• • Compiler hanya memeriksa silsilah: `Manajer` memang turunan dari `Pegawai`, sehingga konversi secara teoretis memungkinkan.

• **Mengapa Terjadi Crash saat Runtime?**

• • Saat dieksekusi, JVM memeriksa objek riil di heap memori: objek tersebut adalah `Programmer`, bukan `Manajer`! Hubungan `Programmer` IS-A `Manajer` adalah SALAH.

• **Aturan Emas Pemrograman Java:**

• • **JANGAN PERNAH** melakukan downcasting tanpa membungkusnya dalam pengecekan **`instanceof`** terlebih dahulu!

Modern Java: Pattern Matching for instanceof

Pendekatan Tradisional (Klasik)

```java
// CARA TRADISIONAL (Java 1.0 s/d Java 15):
// Perlu 2 langkah: pengecekan tipe, lalu casting manual
if (peg instanceof Manajer) {
    Manajer m = (Manajer) peg; // Redundan & verbos
    System.out.println("Tunjangan: " + m.getTunjangan());
}
```

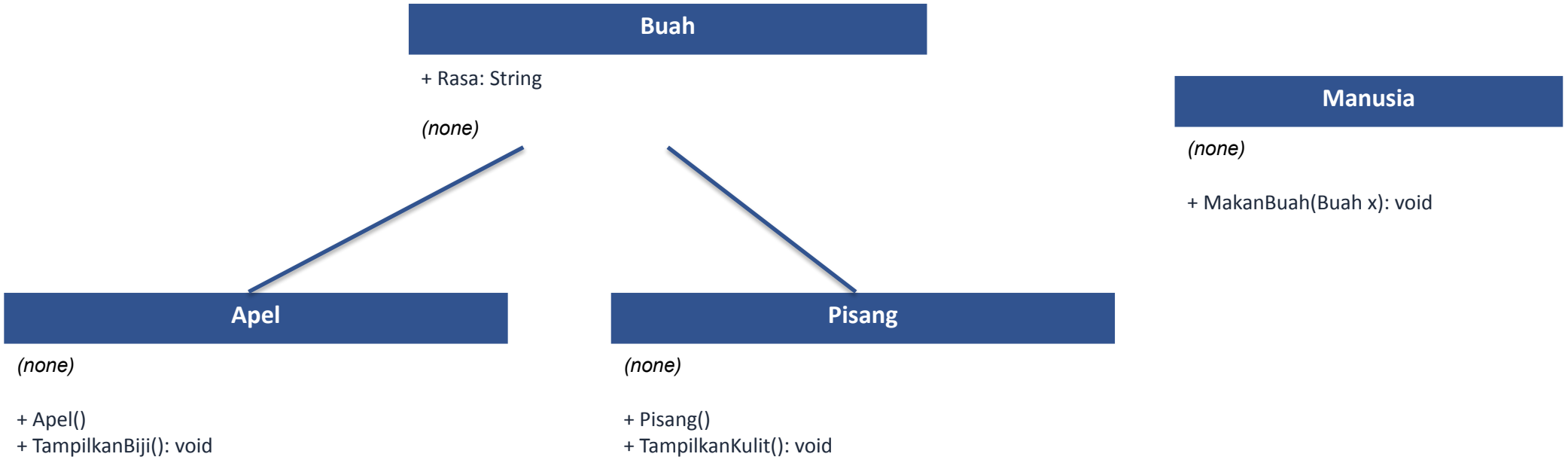
Pendekatan Modern (Java 16+)

```java
// CARA MODERN (Java 16+ via JEP 394):
// Menggabungkan pemeriksaan tipe dan casting dalam 1 ekspresi!
if (peg instanceof Manajer m) {
    // Variabel 'm' langsung tercipta & bertipe Manajer
    System.out.println("Tunjangan: " + m.getTunjangan());
}
```

Keunggulan Pattern Matching

- **Pembersihan Boilerplate:** Mengeliminasi baris casting manual `(Manajer) peg` yang berulang dan rawan typo.
- **Keamanan Ruang Lingkup (Scope Safety):** Variabel pola `m` hanya hidup dan dapat diakses di dalam blok `if` tersebut.
- **Kompatibilitas:** Seluruh kode lama tetap berfungsi penuh, namun fitur ini menjadi standar penulisan Java masa kini.

Studi Kasus: Sistem Buah & Manusia (UML)



Deskripsi Hubungan Sistem

Class Apel dan Pisang adalah turunan dari Buah. Class Manusia memiliki method MakanBuah yang menerima argumen polimorfik bertipe Buah x, lalu memanfaatkan instanceof dan downcasting untuk menampilkan perilaku khas masing-masing buah.

Studi Kasus Buah: Source Code Subclass

Class Buah.java

```
public class Buah {
    public String Rasa;
}
```

Struktur Data

Atribut 'Rasa' dideklarasikan di class induk Buah dan diwarisi ke Apel dan Pisang. Method TampilkanBiji() hanya ada di Apel, sedangkan TampilkanKulit() hanya ada di Pisang.

Class Apel.java

```
public class Apel extends Buah {
    public Apel() {
        Rasa = "manis agak asem";
    }

    public void TampilkanBiji() {
        System.out.println(
            "Biji apel berukuran
kecil");
    }
}
```

Class Pisang.java

```
public class Pisang extends Buah {
    public Pisang() {
        Rasa = "sepet";
    }

    public void TampilkanKulit() {
        System.out.println(
            "Kulit pisang berwarna
kuning");
    }
}
```

Studi Kasus Buah: Logika MakanBuah & Downcasting

Class Manusia.java

```java
public class Manusia {
    public void MakanBuah(Buah x) {
        if (x instanceof Apel) {
            System.out.println("Apel langsung dimakan");
            // Downcasting: x bertipe Buah di-cast ke Apel
            ((Apel) x).TampilkanBiji();
        }
        else if (x instanceof Pisang) {
            System.out.println("Pisang dikupas terlebih dahulu");
            // Downcasting: x bertipe Buah di-cast ke Pisang
            ((Pisang) x).TampilkanKulit();
        }

        // Mengakses atribut Rasa milik superclass Buah
        System.out.println("Rasanya " + x.Rasa);
    }
}
```

Poin Analisis Kunci

- **Pengecekan Tipe:** `if (x instanceof Apel)` mendeteksi apakah objek buah sesungguhnya adalah Apel.
- **Mengapa Perlu Casting?**
- • Variabel `x` bertipe `Buah`, sementara method `TampilkanBiji()` hanya dimiliki oleh class `Apel`.
- • Tanpa casting eksplisit `((Apel) x)`, baris tersebut akan menyebabkan compiler error karena class `Buah` tidak mengenal `TampilkanBiji()`.
- **Akses Superclass:** Baris `x.Rasa` dapat dipanggil langsung tanpa casting karena `Rasa` merupakan member asli dari class `Buah`.

Studi Kasus Buah: Pengujian & Hasil Eksekusi

Class TestBuah.java

```java
public class TestBuah {
    public static void main(String[] args) {
        Manusia ilham = new Manusia();

        Apel myApel = new Apel();
        Pisang myPisang = new Pisang();

        // Mengirim objek Pisang ke method MakanBuah:
        ilham.MakanBuah(myPisang);
    }
}
```

Command Prompt / Terminal

```
D:\Java> javac TestBuah.java
D:\Java> java TestBuah
Pisang dikupas terlebih dahulu
Kulit pisang berwarna kuning
Rasanya sepet
```

Verifikasi Output

- Objek `myPisang` dilewatkan ke parameter `Buah x`.
- Pengecekan `x instanceof Pisang` bernilai `true`.
- Method `TampilkanKulit()` berhasil dieksekusi setelah downcasting.
- Atribut `Rasa` mencetak nilai inisialisasi konstruktor Pisang: *'sepet'*.

Jebakan OOP: Variable Hiding vs Method Overriding

Prinsip Variable Hiding

- **Aturan Mutlak Java:** **Atribut (Variable / Field) TIDAK Bersifat Polimorfik!**
- Jika subclass mendeklarasikan atribut dengan nama yang persis sama dengan atribut superclass, hal itu disebut **Variable Hiding**, bukan overriding.
- Pengaksesan atribut selalu ditentukan oleh **Tipe Referensi** (waktu kompilasi), bukan tipe objek aktual di memori heap!
- Hanya **instance method** yang mengalami *dynamic dispatch* polimorfik.
- **Best Practice:** Selalu jaga atribut berstatus `private` atau `protected`, dan gunakan method getter/setter agar akses data tetap bersifat polimorfik!

Bukti Kode Program

```
class Parent {
    int x = 10;
    void show() { System.out.println("Parent"); }
}
class Child extends Parent {
    int x = 20; // Hiding Parent.x
    @Override
    void show() { System.out.println("Child"); }
}

public class TestHiding {
    public static void main(String[] args) {
        Parent p = new Child();
        System.out.println(p.x); // Output: 10
(Parent.x!)
        p.show();                // Output: Child
(Polimorfik!)
    }
}
```

Method yang Tidak Mendukung Dynamic Polymorphism

Jenis Method	Alasan Tidak Polimorfik Dinamis	Konsekuensi di Java
static method	Terikat pada Class, bukan pada objek instance. Menggunakan static binding saat kompilasi.	Jika ditulis ulang di subclass, disebut 'Method Hiding', bukan overriding.
final method	Kata kunci final secara eksplisit melarang subclass untuk mengubah atau meng-override method.	Compiler akan menolak (error) jika subclass mencoba mendefinisikannya kembali.
private method	Hanya terlihat di dalam class itu sendiri dan tidak diwariskan ke subclass.	Subclass tidak dapat meng-override method yang tidak dapat dilihatnya.

Syarat Mutlak Dynamic Polymorphism

Dynamic Polymorphism HANYA berlaku untuk instance method yang berstatus non-static, non-final, dan memiliki hak akses yang mengizinkan pewarisan (public, protected, atau default/package-private jika dalam package yang sama).

Best Practice: Liskov Substitution Principle (LSP)

Prinsip Desain Bersih (Clean OOP)

• **Prinsip LSP (SOLID):** *'Objek dari superclass harus dapat digantikan oleh objek dari subclass-nya tanpa mengubah kebenaran dan ekspektasi perilaku program.'*

• **Tanda Bahaya Desain (Code Smell) - Overuse instanceof:**

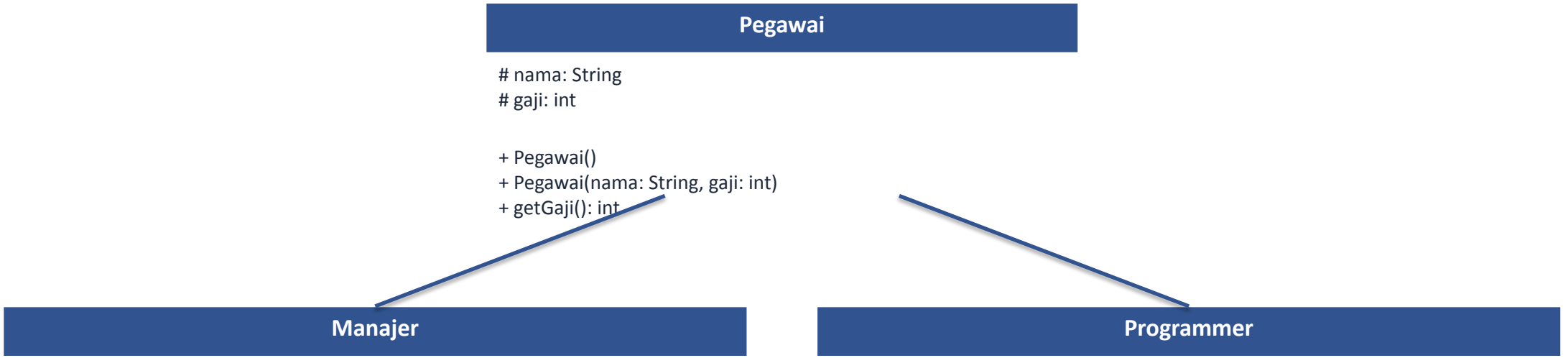
• • Jika Anda menulis rantai `if (x instanceof A) ... else if (x instanceof B) ... else if (x instanceof C) ...` yang terus memanjang setiap kali ada class baru, itu pertanda desain OOP Anda kurang optimal.

• • Menambah class baru mengharuskan Anda memodifikasi kode method lama (melanggar *Open/Closed Principle*).

• **Solusi Desain Bersih:**

• • Pindahkan perilaku spesifik menjadi **abstract method** di superclass, dan biarkan masing-masing subclass meng-overriding logikanya sendiri secara mandiri!

Exercise 1: Sistem Penggajian Pegawai



Instruksi Latihan

Buatlah implementasi class diagram di atas, kemudian buat class Bayaran dan TestBayaran untuk menguji perhitungan gaji pegawai.

Exercise 1: Implementasi Bayaran & TestBayaran

Class Bayaran.java

```
public class Bayaran {
    public int hitungBayaran(Pegawai pg) {
        int uang = pg.getGaji();

        if (pg instanceof Manajer) {
            uang += ((Manajer) pg).getTunjangan();
        }
        else if (pg instanceof Programmer) {
            uang += ((Programmer) pg).getBonus();
        }

        return uang;
    }
}
```

Class TestBayaran.java

```
public class TestBayaran {
    public static void main(String[] args) {
        Manajer man = new Manajer("Agus", 800, 50);
        Programmer prog = new Programmer("Budi", 600,
30);
        Bayaran hr = new Bayaran();

        System.out.println("Bayaran untuk Manajer : "
            + hr.hitungBayaran(man));
        System.out.println("Bayaran untuk Programmer :
"
            + hr.hitungBayaran(prog));
    }
}
```

Exercise 1: Hasil Eksekusi & Evaluasi

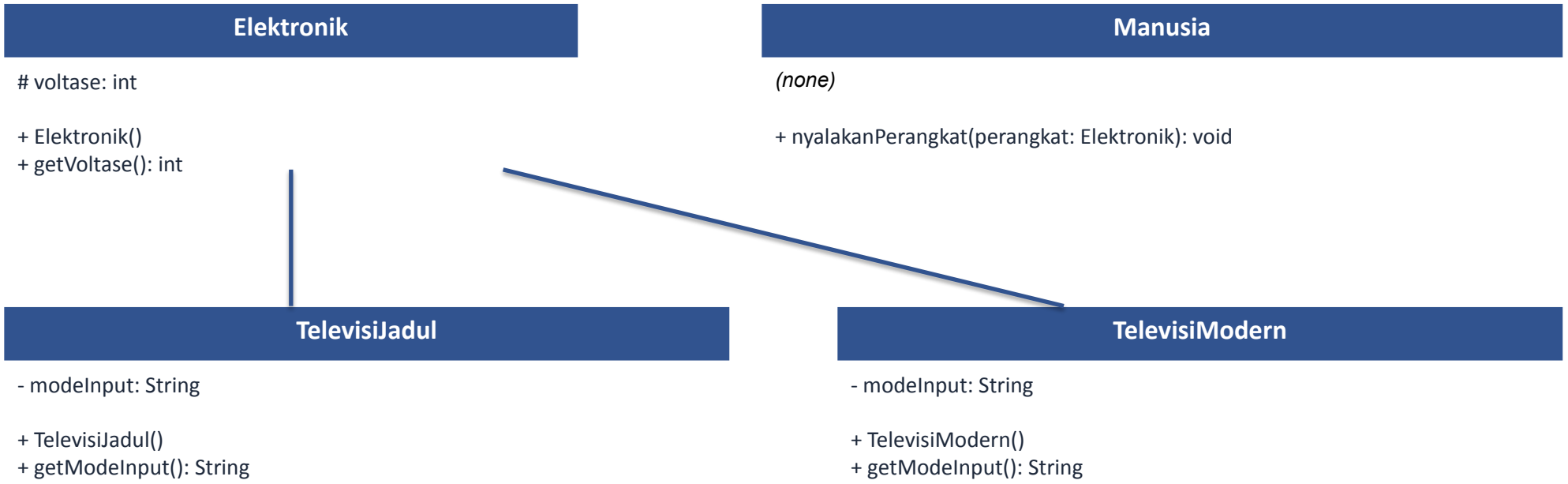
Command Prompt / Terminal

```
D:\Java> javac TestBayaran.java
D:\Java> java TestBayaran
Bayaran untuk Manajer : 850
Bayaran untuk Programmer : 630
```

Pembahasan Logika

- **Polymorphic Parameter:** Method `hitungBayaran(Pegawai pg)` dapat menerima objek `man` (`Manajer`) dan `prog` (`Programmer`) secara fleksibel.
- **Gaji Pokok:** Diambil langsung via `pg.getGaji()` dari superclass `Pegawai`.
- **Tunjangan Tambahan:** Menggunakan `instanceof` untuk verifikasi tipe, lalu di-downcast ke `((Manajer) pg)` untuk mengambil tunjangan (+50) atau ke `((Programmer) pg)` untuk mengambil bonus (+30).
- **Hasil Akhir:** Manajer = 800 + 50 = 850; Programmer = 600 + 30 = 630.

Exercise 2: Perangkat Elektronik



Spesifikasi Latihan

Konstruktor TelevisiJadul mengeset modeInput='DVI' dan voltase=220. Konstruktor TelevisiModern mengeset modeInput='HDMI' dan voltase=220. Method nyalakanPerangkat mendeteksi tipe TV dan menyalakannya.

Exercise 2: Pengujian & Hasil Eksekusi

Class TestElektronik.java

```java
public class TestElektronik {
    public static void main(String[] args) {
        Manusia indro = new Manusia();

        TelevisiJadul tvjadul = new TelevisiJadul();
        TelevisiModern tvmodern = new TelevisiModern();

        indro.nyalakanPerangkat(tvjadul);
        indro.nyalakanPerangkat(tvmodern);
    }
}
```

Command Prompt / Terminal

```
D:\Java\Polimorfisme> javac TestElektronik.java
D:\Java\Polimorfisme> java TestElektronik
Nyalakan televisi jadul dengan input: DVI
Voltase televisi: 220
Nyalakan televisi modern dengan input: HDMI
Voltase televisi: 220
```

Intisari Solusi

- Parameter `perangkat` bertipe umum `Elektronik`.
- Method `nyalakanPerangkat` mengecek `perangkat instanceof TelevisiJadul` dan `TelevisiModern`.
- Nilai `modeInput` diambil via casting `((TelevisiJadul) perangkat).getModeInput()`, sedangkan `voltase` diambil via `perangkat.getVoltase()`.

Rangkuman: Static vs Dynamic Polymorphism

Dimensi	Static Polymorphism	Dynamic Polymorphism
Mekanisme Utama	Method Overloading	Method Overriding
Waktu Pengikatan	Compile-Time (Static / Early Binding)	Run-Time (Dynamic / Late Binding)
Penentu Eksekusi	Signature method (tipe & jumlah parameter)	Tipe objek riil di heap memori via vtable
Syarat Pewarisan	Tidak butuh Inheritance (cukup dalam 1 class)	Wajib ada relasi Inheritance atau Interface
Pembeda Return Type	Tidak boleh berbeda hanya pada return type	Boleh kovarian return type pada subclass
Fitur Terkait	Banyak method dengan nama sama	Virtual Method Invocation, instanceof, Casting

Polimorfisme mengubah cara pandang kita dalam menyusun program: dari pemrograman yang kaku dan terikat pada tipe konkret, menjadi sistem modular yang fleksibel, mudah dikembangkan, dan siap beradaptasi dengan perubahan kebutuhan perangkat lunak di masa depan.

To be continued...

Langkah Selanjutnya

- Materi polimorfisme merupakan pondasi esensial sebelum melangkah ke konsep tingkat lanjut:
- • **Abstract Class**: Memaksa standardisasi method polimorfik tanpa implementasi default.
- • **Interface**: Fondasi arsitektur modular dan *Dependency Injection* dalam framework modern.
- • **Design Patterns**: Pola desain seperti Factory, Strategy, dan State Pattern dibangun di atas pilar polimorfisme.
- Kerjakan Jobsheet 10 untuk memantapkan pemahaman praktis Anda di laboratorium pemrograman!

Ada Pertanyaan? Silakan Berdiskusi!

Terima kasih atas perhatian dan antusiasme Anda dalam perkuliahan hari ini.
Tetap semangat belajar, bereksperimen, dan terus mengasah kemampuan coding Anda!
Team Teaching PBO - Jurusan Teknologi Informasi, Politeknik Negeri Malang